Renaming saturation arithmetic functions

Document number:
P4052R0

Date:
2026-03-13

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>
Corentin Jabot <corentin.jabot@gmail.com>

GitHub Issue:
wg21.link/P4052/github

Source:
github.com/eisenwave/cpp-proposals/blob/master/src/rename-sat.cow

---

ABSTRACT: Saturation arithmetic functions should be renamed. This paper resolves NB comment [FR-026-265].

§ Contents

§ 1. Introduction

NB comment [FR-026-265] states the following:

> The names `add_sat`, `sub_sat`, `mul_sat` etc are rather confusing and nondescript
>
> **Proposed change:** Rename to `saturating_add`, `saturating_sub`, `saturating_mul`, `saturating_div` OR Rename to `add_saturate` / `sub_saturate` / `mul_saturate` / `div_saturate`.

[P0543R3] provides the following rationale for the status quo:

> ## Naming
>
> Considerations:
>
> - These are basic low-level operations. Names should be short.
> - `add` / `sub` / `mul` / `div` are common abbreviation for the corresponding arithmetic operations.
>
> Options (only showing the operation "saturated addition"):
>
> - `addsat`, `satadd`: not easily readable
> - `saturated_add`, `add_saturated`: too long
> - `sat_add`: more readable due to underscore
> - `add_sat`: more readable due to underscore, precedence in OpenCL
>
> The last choice is taken.

1.1. Arguments against `sat`

The NB comment is right in the sense that to a novice, the abbreviation `sat` does not have obvious meaning. It can also be argued not to match the recommendation of the [LEWGDesignGuidelines]:

> Avoid abbreviations except for common words: `_ptr`, `std`, etc. (apply common sense).

While conciseness is generally desirable in numeric context, it may have been given too much importance in [P0543R3]. Several arguments speak against the status quo:

- The status quo is not internally consistent. `add_sat` has a heavily abbreviated name, whereas `saturate_cast` is unabbreviated. By comparison, there is a Rust crate `saturating_cast`, which is internally consistent with `saturating_add` in the Rust standard library.
- Many other libraries do not abbreviate `sat`:

Library	Function
Rust Standard Library	`saturating_add`

Java, Guava Core Libraries	`saturatedAdd`
C#, .NET	`AddSaturate`
C++, LLVM	`SaturatingAdd`

- At the time of writing, GitHub code search yields the following results:

Search Reg. Exp.	# Files
`\bsaturating_add\b`	204K
`\badd_sat\b`	71.9K
`\bsat_add\b`	3.1K
`\badd_saturate\b`	1.1K
`\bsaturated_add\b`	738
`\badd_saturated\b`	656
`\bsaturate_add\b`	248
`\badd_saturating\b`	189
`\badd_saturation\b`	143
`\bsaturation_add\b`	55

  If `sat` must be abbreviated because the alternative is "too long", why is the unabbreviated `saturating_add` almost three times as popular?

- Both "`std::add_sat(x, y)`" (18 characters) and "`std::saturating_add(x, y)`" (25 characters) are verbose, compared to "`x + y`" (5 characters). If large amounts of operations need to be made saturating, the user should create a class template with saturating `operator+`.

§ 1.2. Broader design context

The naming of saturation arithmetic functions is hugely important because it essentially sets the naming policy for all future "arithmetic variations". For example, Rust supports a large amount of variations of multiplication:

Rust function	Meaning	C++ counterpart
`saturating_mul`	Returns product clamped to numeric range of type	C++26 `std::mul_sat`
`overflowing_mul`	Returns product, as well as `bool` indicating overflow	[P3161R4] `std::mul_overflow`, C++26 and C23: `ckd_mul`, GCC: `__builtin_mul_overflow`
`widening_mul`	Returns low and high bits of product	[P3161R4] `std::mul_wide`
`wrapping_mul`	Wrapping multiplication (including for signed types)	`unsigned` multiplication

`checked_mul`	Returns `Option<T>` containing product, or empty result on overflow	N/A

It is plausible that given time, some or all of these variations of multiplication could be available in C++. We thus need a sustainable naming scheme that can handle all such variations.

> NOTE: Also related is [P3642R4], which proposes `std::clmul_wide` as the widening variation of `std::clmul`. Note that the naming scheme here deliberately imitates that of `std::mul_sat`; it is not "convergent evolution".

2. Proposal

The existing functions in [numeric.sat] should all be renamed to follow a `saturating_`*op* naming scheme, including `std::saturate_cast`. This should be done for the following reasons:

- The meaning of `saturating` is more obvious than for `sat`.
- The design is familiar to Rust users, and a similar Rust-based approach can be taken for many more operation variations.
- The naming scheme is internally consistent, unlike `add_sat` vs. `saturate_cast`.
- `saturating_`*op* is the most common naming scheme. Any other option (`sat`, `saturate`, etc.) yields fewer results in GitHub code search.

Furthermore, LEWG should commit to the Rust naming scheme for future proposals, such as [P3161R4] and [P3642R4], which propose `std::mul_wide` and `std::clmul_wide`, respectively.

3. Wording

The changes are relative to [N5032].

Bump the feature-test macro in [version.syn] as follows:

```
#define __cpp_lib_saturation_arithmetic 202311L 202603L // freestanding, also in
```

> DRAFTING NOTE: Without bumping the feature test macro, `__cpp_lib_saturation_arithmetic >= 202311L` could be `true`, with `std::saturating_add` being ill-formed.

Change [numeric.ops.overview] as follows:

```
namespace std {
  […]

  // [numeric.sat], saturation arithmetic
  template<class T>
    constexpr T ~~add_sat~~ saturating_add(T x, T y) noexcept;
  template<class T>
    constexpr T ~~sub_sat~~ saturating_sub(T x, T y) noexcept;
  template<class T>
    constexpr T ~~mul_sat~~ saturating_mul(T x, T y) noexcept;
  template<class T>
    constexpr T ~~div_sat~~ saturating_div(T x, T y) noexcept;
  template<class T, class U>
    constexpr T ~~saturate_cast~~ saturating_cast(U x) noexcept;
}
```

Rename the declarations in [numeric.sat] analogously.

§ 4. References

[N5032] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-12-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf

[P0543R3] *Jens Maurer.* Saturation arithmetic (2023-07-19)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0543r3.html

[P3161R4] *Tiago Freire.* Unified integer overflow arithmetic (2026-03-12)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3161r4.html

[P3642R4] *Jan Schulte.* Carry-less product: std::clmul (2026-02-17)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3642r4.html

[FR-026-265] *AFNOR.* Confusing names add_sat, sub_sat, mul_sat (2025-10-03)
https://github.com/cplusplus/nbballot/issues/840

[LEWGDesignGuidelines] (2018-04-16)
https://github.com/cplusplus/LEWG/blob/archive/library-design-guidelines.md